ShiftSwap

Hackathon problem statement

Background

CityCare Clinic runs a weekly staff roster. When someone cannot work a shift, they post it for a colleague to take over, and a manager approves the change. Build the system that manages these swaps and protects the clinic's labour rules.

What to build

A web application (UI and backend) where staff post and claim shifts and managers approve or reject claims. Every action is validated against the rules below, and every refusal tells the user why.

Data

`seed.json` is provided with staff and the current roster. Load it only when your app has no saved state. The roster in it already satisfies every rule below.

```
staff:  { id, name, role ("STAFF" | "MANAGER"), certifications: [string] }
shifts: { id, start, end, required_certification (string | null), assigned_to }
```

- Times are local clinic time as `YYYY-MM-DDTHH:MM`. There are no time zones or daylight saving. A shift may cross midnight.
- Managers have no shifts and cannot post or claim. Only managers approve or reject.
- Treat every shift as upcoming. The real current date and time must not affect any rule.

Definitions

- Two shifts **overlap** if A.start < B.end and B.start < A.end.
- A **week** runs from Monday 00:00 to the next Monday 00:00. A shift belongs to the week in which it starts.
- A shift counts as assigned to its current person until a swap for it is approved, including while it is listed. A pending claim does not count as an assignment.

Listings

A listing is a request to give away one shift. A shift can have at most one listing that is OPEN or CLAIMED at a time.

Action	Who	Allowed when	Result
Post	Shift owner	Shift has no OPEN or CLAIMED listing	Listing OPEN
Withdraw	Poster	Listing OPEN	Listing WITHDRAWN
Claim	Other staff	Listing OPEN and the claim rules pass	Listing CLAIMED by them
Unclaim	Claimer	Listing CLAIMED	Listing OPEN
Approve	Manager	Listing CLAIMED and the claim rules still pass	Shift reassigned, listing APPROVED
Reject	Manager	Listing CLAIMED	Listing OPEN

Claim rules

Staff member C claims shift S, owned by P. All boundaries are inclusive (meeting a limit exactly is allowed).

#	Rule
1	C is not P.
2	If S requires a certification, C has it.
3	S does not overlap any shift assigned to C.
4	For every shift X assigned to C that does not overlap S, at least 11 hours separate the end of the earlier shift from the start of the later one.
5	C's total hours in the week in which S starts, including S, are at most 40.

- If the listing is no longer OPEN, refuse with "no longer available" and check nothing else.
- Otherwise report **every** violated rule, each identifying the rule and, where relevant, the conflicting shift. If C is P, report only rule 1.
- On approval, re-check rules 2 to 5 against C's current assignments. If any fail, refuse the approval with the reasons and leave the listing CLAIMED.

Required features

- Choose which person you are acting as. There is no login.
- Staff: see own shifts and which are listed. Post and withdraw. See other people's OPEN listings with whether you can claim each one and, if not, all reasons. Claim and unclaim.
- Managers: see CLAIMED listings. Approve and reject.
- Everyone: a history of actions (who, what, which shift, outcome).
- State survives a restart of your backend.

General instructions

- You have **3 hours** from receiving this document. The deadline is strict.
- Use any language, framework, database and AI tools. The project must run locally without paid services or API keys.
- Submit a **GitHub repo link** through [submission form / email] by the deadline. The repo can be public, or private and shared with [GitHub handle].
- The repo must contain your code, a `README.md` (what you built, exact steps to run it, your assumptions) and an `AI_USAGE.md` (where you used AI and what you accepted, changed or rejected).
- **No commits after the end time.** Late commits will not be considered.
- **Bonus (optional):** a link to your app deployed on any free hosting platform, sent through the same channel within 30 minutes after the deadline. Do not commit it to the repo. Deploy from your final commit.
- Work individually. Do not share this problem or your solution with anyone, during or after the event.
- If something is unclear, make a reasonable assumption and record it in your README. Additional requirements may be announced during the event.
- You may be invited to a call after the event to explain your submission.